FLET + FASTAPI: THE COMPLETE FULL-STACK GUIDE

Connecting Frontend, Backend, and Everything In Between

Table of Contents

1. [Introduction to Full-Stack Development with Flet & FastAPI](#1-introduction-to-full-stack-development-with-flet--fastapi)
2. [Flet + FastAPI Integration](#2-flet--fastapi-integration)
3. [Database Integration & ORMs](#3-database-integration--orms)
4. [Authentication & Security](#4-authentication--security)
5. [Background Tasks & Real-Time Events](#5-background-tasks--real-time-events)
6. [Packaging & Deployment](#6-packaging--deployment)
7. [Complete Full-Stack Application Example](#7-complete-full-stack-application-example)

---

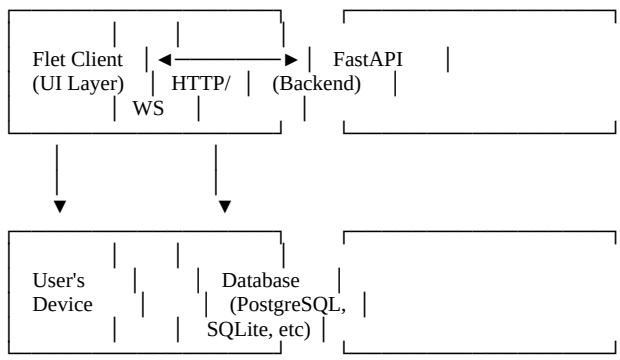
1. Introduction to Full-Stack Development with Flet & FastAPI

Why Flet + FastAPI?

Flet and **FastAPI** form a powerful full-stack combination:

Feature	Flet (Frontend)	FastAPI (Backend)
Purpose	Cross-platform UI (web, desktop, mobile)	High-performance API server
Language	Python	Python
Architecture	Flutter-based rendering	ASGI asynchronous server
Key Strength	One codebase, all platforms	Speed, automatic docs, validation
Communication	HTTP/WebSocket client	REST/WebSocket server

The Communication Flow



Project Structure for Full-Stack Apps

```
my_fullstack_app/
├── frontend/
│   ├── main.py             # Flet app entrypoint
│   ├── views/
│   │   ├── __init__.py
│   │   ├── home.py
│   │   ├── login.py
│   │   └── dashboard.py
│   ├── components/
│   │   ├── __init__.py
│   │   ├── buttons.py
│   │   └── dialogs.py
│   └── services/
│       ├── __init__.py
│       ├── api_client.py    # FastAPI communication
│       └── auth_service.py
│
├── backend/
```

```
│   ├── __init__.py
│   ├── main.py              # FastAPI app entrypoint
│   ├── models/
│   │   ├── __init__.py
│   │   ├── user.py
│   │   └── item.py
│   ├── schemas/
│   │   ├── __init__.py
│   │   ├── user.py
│   │   └── item.py
│   ├── database/
│   │   ├── __init__.py
│   │   ├── session.py
│   │   └── migrations/
│   ├── routers/
│   │   ├── __init__.py
│   │   ├── users.py
│   │   └── items.py
│   └── security/
│       ├── __init__.py
│       ├── auth.py
│       └── jwt.py
│
├── shared/
│   ├── __init__.py
│   └── models.py            # Shared Pydantic models
│
├── pyproject.toml
├── docker-compose.yml
└── .env
```

---

2. Flet + FastAPI Integration

Option 1: Mounting Flet Inside FastAPI (Dynamic Website)

This approach serves your Flet app as a dynamic website through FastAPI's ASGI server.

Setup

```python
# backend/main.py
import flet as ft
import flet.fastapi as flet_fastapi
from fastapi import FastAPI
from contextlib import asynccontextmanager

# Define your Flet app
async def flet_main(page: ft.Page):
    page.title = "My Full-Stack App"

    # Fetch data from the API
    response = await page.client_storage.get("api_data") or {"message": "Hello from backend!"}

    page.add(
        ft.Text("Flet + FastAPI Full-Stack App", size=32, weight=ft.FontWeight.BOLD),
        ft.Text(f"Data from API: {response.get('message', 'No data')}"),
        ft.Button("Fetch Data", on_click=lambda e: fetch_data(page)),
        ft.TextField(label="Enter text", ref=text_ref),
        ft.Button("Send to Backend", on_click=lambda e: send_data(page)),
    )

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Start Flet app manager
    await flet_fastapi.app_manager.start()
    yield
    # Shutdown
    await flet_fastapi.app_manager.shutdown()

# Create FastAPI app
```

```
app = FastAPI(lifespan=lifespan)

# Mount Flet app
app.mount("/", flet_fastapi.app(flet_main))

# Add API endpoints
@app.get("/api/data")
async def get_data():
    return {"message": "Hello from FastAPI!", "timestamp": datetime.now().isoformat()}

@app.post("/api/data")
async def post_data(data: dict):
    # Process data from Flet
    return {"status": "success", "received": data}
```

Running the App

```bash
# Install dependencies
uv add "flet[fastapi]" fastapi uvicorn

# Run with Uvicorn
uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

Option 2: Separate Flet and FastAPI Services

This approach runs Flet and FastAPI as separate services, communicating via HTTP.

FastAPI Backend

```python
# backend/main.py
from fastapi import FastAPI, HTTPException, Depends
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel

app = FastAPI()

# CORS for Flet web app
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:8550", "http://localhost:8000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class ItemCreate(BaseModel):
    name: str
    description: str | None = None
    price: float

class ItemResponse(BaseModel):
    id: int
    name: str
    description: str | None = None
    price: float

items_db = {}
item_counter = 0

@app.post("/api/items", response_model=ItemResponse)
async def create_item(item: ItemCreate):
    global item_counter
    item_counter += 1
    new_item = ItemResponse(id=item_counter, **item.model_dump())
    items_db[item_counter] = new_item
    return new_item

@app.get("/api/items", response_model=list[ItemResponse])
async def get_items():
```

```
    return list(items_db.values())

@app.get("/api/items/{item_id}", response_model=ItemResponse)
async def get_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(status_code=404, detail="Item not found")
    return items_db[item_id]

@app.delete("/api/items/{item_id}")
async def delete_item(item_id: int):
    if item_id not in items_db:
        raise HTTPException(status_code=404, detail="Item not found")
    del items_db[item_id]
    return {"status": "deleted"}

# Run with: uvicorn backend.main:app --reload --port 8000
```

Flet Frontend with HTTP Client

```python
# frontend/main.py
import flet as ft
import httpx
import asyncio

API_URL = "http://localhost:8000/api"

class ApiService:
    @staticmethod
    async def get_items():
        async with httpx.AsyncClient() as client:
            response = await client.get(f"{API_URL}/items")
            response.raise_for_status()
            return response.json()

    @staticmethod
    async def create_item(name: str, description: str, price: float):
        async with httpx.AsyncClient() as client:
            response = await client.post(
                f"{API_URL}/items",
                json={"name": name, "description": description, "price": price}
            )
            response.raise_for_status()
            return response.json()

    @staticmethod
    async def delete_item(item_id: int):
        async with httpx.AsyncClient() as client:
            response = await client.delete(f"{API_URL}/items/{item_id}")
            response.raise_for_status()
            return response.json()

def main(page: ft.Page):
    page.title = "Flet + FastAPI Example"
    page.scroll = ft.ScrollMode.AUTO

    items_ref = ft.Ref[ft.Column]()
    name_ref = ft.Ref[ft.TextField]()
    desc_ref = ft.Ref[ft.TextField]()
    price_ref = ft.Ref[ft.TextField]()
    status_ref = ft.Ref[ft.Text]()

    async def load_items():
        try:
            items = await ApiService.get_items()
            items_ref.current.controls.clear()
            for item in items:
                items_ref.current.controls.append(
                    ft.Row([
                        ft.Text(f"{item['name']} - ${item['price']}"),
                        ft.Text(item['description'] or ""),
                        ft.IconButton(
```

```
                        ft.Icons.DELETE,
                        on_click=lambda e, item_id=item['id']: delete_item(item_id)
                    ),
                ])
            )
        page.update()
    except Exception as e:
        status_ref.current.value = f"Error loading items: {e}"
        page.update()

async def add_item(e):
    name = name_ref.current.value
    description = desc_ref.current.value
    try:
        price = float(price_ref.current.value)
    except ValueError:
        status_ref.current.value = "Invalid price"
        page.update()
        return

    if not name:
        status_ref.current.value = "Name is required"
        page.update()
        return

    try:
        await ApiService.create_item(name, description, price)
        name_ref.current.value = ""
        desc_ref.current.value = ""
        price_ref.current.value = ""
        status_ref.current.value = "Item added successfully!"
        await load_items()
    except Exception as e:
        status_ref.current.value = f"Error adding item: {e}"
    page.update()

async def delete_item(item_id: int):
    try:
        await ApiService.delete_item(item_id)
        status_ref.current.value = "Item deleted!"
        await load_items()
    except Exception as e:
        status_ref.current.value = f"Error deleting item: {e}"
    page.update()

# Build UI
page.add(
    ft.Text("Flet + FastAPI CRUD Example", size=32, weight=ft.FontWeight.BOLD),
    ft.Text("Items", size=24),
    ft.Column(ref=items_ref),
    ft.Divider(),
    ft.Text("Add New Item", size=20),
    ft.Row([
        ft.TextField(ref=name_ref, label="Name", width=200),
        ft.TextField(ref=desc_ref, label="Description", width=200),
        ft.TextField(ref=price_ref, label="Price", width=150, keyboard_type=ft.KeyboardType.NUMBER),
    ]),
    ft.Row([
        ft.Button("Add Item", on_click=lambda e: asyncio.create_task(add_item(e))),
        ft.Button("Refresh", on_click=lambda e: asyncio.create_task(load_items())),
    ]),
    ft.Text(ref=status_ref),
)

# Initial load
asyncio.create_task(load_items())

ft.run(main, view=ft.WEB_BROWSER, port=8550)
```

Option 3: WebSocket Communication for Real-Time Updates

FastAPI WebSocket Server

```python
# backend/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from typing import List

app = FastAPI()

class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []

    async def connect(self, websocket: WebSocket):
        await websocket.accept()
        self.active_connections.append(websocket)

    def disconnect(self, websocket: WebSocket):
        self.active_connections.remove(websocket)

    async def broadcast(self, message: str):
        for connection in self.active_connections:
            await connection.send_text(message)

manager = ConnectionManager()

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await manager.connect(websocket)
    try:
        while True:
            data = await websocket.receive_text()
            # Process message
            await manager.broadcast(f"Message: {data}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)
```

Flet WebSocket Client

```python
# frontend/main.py
import flet as ft
import websockets
import asyncio

class WebSocketService:
    def __init__(self, page: ft.Page):
        self.page = page
        self.websocket = None
        self.messages = []

    async def connect(self):
        try:
            self.websocket = await websockets.connect("ws://localhost:8000/ws")
            asyncio.create_task(self.listen())
            return True
        except Exception as e:
            print(f"Connection error: {e}")
            return False

    async def listen(self):
        try:
            while True:
                message = await self.websocket.recv()
                self.messages.append(message)
                self.page.update()
        except websockets.exceptions.ConnectionClosed:
            print("Connection closed")

    async def send(self, message: str):
        if self.websocket:
            await self.websocket.send(message)
```

```
    async def disconnect(self):
        if self.websocket:
            await self.websocket.close()

def main(page: ft.Page):
    page.title = "WebSocket Chat"

    ws = WebSocketService(page)
    messages_ref = ft.Ref[ft.Column]()
    input_ref = ft.Ref[ft.TextField]()
    status_ref = ft.Ref[ft.Text]()

    async def connect_to_ws(e):
        status_ref.current.value = "Connecting..."
        page.update()
        if await ws.connect():
            status_ref.current.value = "Connected!"
            # Start receiving messages
        else:
            status_ref.current.value = "Connection failed"
        page.update()

    async def send_message(e):
        text = input_ref.current.value
        if text and ws.websocket:
            await ws.send(text)
            input_ref.current.value = ""
            page.update()

    def update_messages():
        if messages_ref.current:
            messages_ref.current.controls.clear()
            for msg in ws.messages[-20:]:  # Show last 20 messages
                messages_ref.current.controls.append(ft.Text(msg))
            messages_ref.current.update()

    # Periodic UI update
    def update_loop():
        update_messages()
        page.update()

    page.add(
        ft.Text("WebSocket Chat", size=32),
        ft.Row([
            ft.Button("Connect", on_click=lambda e: asyncio.create_task(connect_to_ws(e))),
            ft.Text(ref=status_ref, value="Disconnected"),
        ]),
        ft.Column(ref=messages_ref, height=300, scroll=ft.ScrollMode.AUTO),
        ft.Row([
            ft.TextField(ref=input_ref, hint_text="Type a message...", expand=True),
            ft.Button("Send", on_click=lambda e: asyncio.create_task(send_message(e))),
        ]),
    )

    # Update loop
    async def loop():
        while True:
            await asyncio.sleep(0.5)
            update_messages()

    asyncio.create_task(loop())

ft.run(main, view=ft.WEB_BROWSER, port=8550)
```

Flet Async Programming Best Practices

```python
# frontend/services/api_client.py
import httpx
import asyncio
from typing import Optional
```

```python
class APIClient:
    def __init__(self, base_url: str):
        self.base_url = base_url
        self.client: Optional[httpx.AsyncClient] = None

    async def __aenter__(self):
        self.client = httpx.AsyncClient(
            base_url=self.base_url,
            timeout=30.0,
            headers={"Content-Type": "application/json"}
        )
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        if self.client:
            await self.client.aclose()

    async def get(self, endpoint: str, params: dict = None):
        response = await self.client.get(endpoint, params=params)
        response.raise_for_status()
        return response.json()

    async def post(self, endpoint: str, data: dict = None):
        response = await self.client.post(endpoint, json=data)
        response.raise_for_status()
        return response.json()

    async def put(self, endpoint: str, data: dict = None):
        response = await self.client.put(endpoint, json=data)
        response.raise_for_status()
        return response.json()

    async def delete(self, endpoint: str):
        response = await self.client.delete(endpoint)
        response.raise_for_status()
        return response.json()

# Usage in Flet
async def fetch_user_data(page: ft.Page, user_id: int):
    async with APIClient("http://localhost:8000/api") as client:
        return await client.get(f"/users/{user_id}")
```

---

3. Database Integration & ORMs

SQLAlchemy 2.0 with Async Support

Setup

```bash
uv add sqlalchemy asyncpg psycopg2-binary
```

Database Configuration

```python
# backend/database/session.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
from sqlalchemy.orm import declarative_base
import os

# PostgreSQL (async)
DATABASE_URL = os.getenv("DATABASE_URL", "postgresql+asyncpg://user:password@localhost:5432/mydb")

# SQLite (async)
# DATABASE_URL = "sqlite+aiosqlite:///./app.db"

engine = create_async_engine(
    DATABASE_URL,
    echo=True,
    pool_size=10,
```

```
    max_overflow=20,
    pool_pre_ping=True,
    pool_recycle=3600,
)

AsyncSessionLocal = async_sessionmaker(
    engine,
    class_=AsyncSession,
    expire_on_commit=False,
    autocommit=False,
    autoflush=False,
)

Base = declarative_base()

async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
        finally:
            await session.close()
```

Models

```python
# backend/models/user.py
from sqlalchemy import Column, Integer, String, DateTime, Boolean, Float, ForeignKey
from sqlalchemy.orm import relationship
from sqlalchemy.sql import func
from backend.database.session import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    username = Column(String(50), unique=True, index=True, nullable=False)
    email = Column(String(100), unique=True, index=True, nullable=False)
    hashed_password = Column(String(255), nullable=False)
    full_name = Column(String(100))
    is_active = Column(Boolean, default=True)
    is_superuser = Column(Boolean, default=False)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, onupdate=func.now())

    # Relationships
    items = relationship("Item", back_populates="owner")

class Item(Base):
    __tablename__ = "items"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    description = Column(String(500))
    price = Column(Float, nullable=False)
    owner_id = Column(Integer, ForeignKey("users.id"))
    created_at = Column(DateTime, server_default=func.now())

    # Relationships
    owner = relationship("User", back_populates="items")
```

Pydantic Schemas

```python
# backend/schemas/user.py
from pydantic import BaseModel, EmailStr, Field
from datetime import datetime
from typing import Optional, List

class UserBase(BaseModel):
    username: str = Field(..., min_length=3, max_length=50)
    email: EmailStr
```

```
    full_name: Optional[str] = None

class UserCreate(UserBase):
    password: str = Field(..., min_length=8)

class UserUpdate(UserBase):
    password: Optional[str] = None
    is_active: Optional[bool] = None

class UserResponse(UserBase):
    id: int
    is_active: bool
    is_superuser: bool
    created_at: datetime

    class Config:
        from_attributes = True

# backend/schemas/item.py
class ItemBase(BaseModel):
    name: str = Field(..., min_length=1, max_length=100)
    description: Optional[str] = None
    price: float = Field(..., gt=0)

class ItemCreate(ItemBase):
    pass

class ItemUpdate(ItemBase):
    name: Optional[str] = None
    price: Optional[float] = None

class ItemResponse(ItemBase):
    id: int
    owner_id: int
    created_at: datetime

    class Config:
        from_attributes = True
```

CRUD Operations

```python
# backend/routers/users.py
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from typing import List

from backend.database.session import get_db
from backend.models.user import User
from backend.schemas.user import UserCreate, UserUpdate, UserResponse
from backend.security.auth import get_password_hash

router = APIRouter(prefix="/users", tags=["users"])

@router.post("/", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
async def create_user(user_data: UserCreate, db: AsyncSession = Depends(get_db)):
    # Check if user exists
    result = await db.execute(select(User).where(User.email == user_data.email))
    if result.scalar_one_or_none():
        raise HTTPException(status_code=400, detail="Email already registered")

    result = await db.execute(select(User).where(User.username == user_data.username))
    if result.scalar_one_or_none():
        raise HTTPException(status_code=400, detail="Username already taken")

    # Create user
    hashed_password = get_password_hash(user_data.password)
    db_user = User(
        username=user_data.username,
        email=user_data.email,
        hashed_password=hashed_password,
```

```
        full_name=user_data.full_name,
    )
    db.add(db_user)
    await db.commit()
    await db.refresh(db_user)
    return db_user

@router.get("/", response_model=List[UserResponse])
async def get_users(
    skip: int = 0,
    limit: int = 100,
    db: AsyncSession = Depends(get_db)
):
    result = await db.execute(select(User).offset(skip).limit(limit))
    return result.scalars().all()

@router.get("/{user_id}", response_model=UserResponse)
async def get_user(user_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == user_id))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user

@router.put("/{user_id}", response_model=UserResponse)
async def update_user(user_id: int, user_data: UserUpdate, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == user_id))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    update_data = user_data.model_dump(exclude_unset=True)
    if "password" in update_data:
        update_data["hashed_password"] = get_password_hash(update_data.pop("password"))

    for field, value in update_data.items():
        setattr(user, field, value)

    await db.commit()
    await db.refresh(user)
    return user

@router.delete("/{user_id}")
async def delete_user(user_id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == user_id))
    user = result.scalar_one_or_none()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")

    await db.delete(user)
    await db.commit()
    return {"status": "deleted"}
```

SQLModel (FastAPI + SQLAlchemy + Pydantic)

```bash
uv add sqlmodel
```

```python
# backend/models/sqlmodel_models.py
from sqlmodel import SQLModel, Field, Relationship
from datetime import datetime
from typing import Optional, List

class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str = Field(max_length=50, unique=True, index=True)
    email: str = Field(max_length=100, unique=True, index=True)
    hashed_password: str = Field(max_length=255)
    full_name: str | None = None
    is_active: bool = Field(default=True)
```

```
    created_at: datetime = Field(default_factory=datetime.now)

    items: List["Item"] = Relationship(back_populates="owner")

class Item(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(max_length=100)
    description: str | None = None
    price: float
    owner_id: int | None = Field(default=None, foreign_key="user.id")
    created_at: datetime = Field(default_factory=datetime.now)

    owner: User | None = Relationship(back_populates="items")

# Schemas
class UserCreate(SQLModel):
    username: str
    email: str
    password: str
    full_name: str | None = None

class UserResponse(SQLModel):
    id: int
    username: str
    email: str
    full_name: str | None = None
    is_active: bool
    created_at: datetime

# Router
from fastapi import Depends, HTTPException
from sqlmodel import Session, select

def get_session():
    with Session(engine) as session:
        yield session

@router.post("/users", response_model=UserResponse)
def create_user(user: UserCreate, session: Session = Depends(get_session)):
    db_user = User.model_validate(user)
    session.add(db_user)
    session.commit()
    session.refresh(db_user)
    return db_user
```

Alembic Database Migrations

```bash
# Install Alembic
uv add alembic

# Initialize Alembic
alembic init alembic

# Configure alembic.ini
# sqlalchemy.url = postgresql://user:password@localhost:5432/mydb

# Generate migration
alembic revision --autogenerate -m "create users and items tables"

# Apply migration
alembic upgrade head

# Rollback
alembic downgrade -1
```

```python
# alembic/env.py
from logging.config import fileConfig
from sqlalchemy import engine_from_config, pool
from alembic import context
```

```
import sys
from pathlib import Path

# Add project path
sys.path.insert(0, str(Path(__file__).parent.parent))
from backend.database.session import Base
from backend.models import user, item

target_metadata = Base.metadata

def run_migrations_offline():
    url = config.get_main_option("sqlalchemy.url")
    context.configure(
        url=url,
        target_metadata=target_metadata,
        literal_binds=True,
        dialect_opts={"paramstyle": "named"},
    )
    with context.begin_transaction():
        context.run_migrations()

def run_migrations_online():
    connectable = engine_from_config(
        config.get_section(config.config_ini_section),
        prefix="sqlalchemy.",
        poolclass=pool.NullPool,
    )
    with connectable.connect() as connection:
        context.configure(
            connection=connection,
            target_metadata=target_metadata,
        )
        with context.begin_transaction():
            context.run_migrations()
```

---

4. Authentication & Security

JWT Authentication System

JWT Utilities

```python
# backend/security/jwt.py
import jwt
from datetime import datetime, timedelta, timezone
from typing import Optional, Dict, Any
from fastapi import HTTPException, status

SECRET_KEY = "your-secret-key-here"  # Store in environment variables
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
REFRESH_TOKEN_EXPIRE_DAYS = 7

def create_access_token(data: Dict[str, Any], expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    if expires_delta:
        expire = datetime.now(timezone.utc) + expires_delta
    else:
        expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire, "type": "access"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def create_refresh_token(data: Dict[str, Any]):
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(days=REFRESH_TOKEN_EXPIRE_DAYS)
    to_encode.update({"exp": expire, "type": "refresh"})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decode_token(token: str) -> Dict[str, Any]:
    try:
```

```
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except jwt.ExpiredSignatureError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Token has expired",
            headers={"WWW-Authenticate": "Bearer"},
        )
    except jwt.InvalidTokenError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token",
            headers={"WWW-Authenticate": "Bearer"},
        )
```

Password Hashing

```python
# backend/security/auth.py
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)

async def authenticate_user(db, username: str, password: str):
    # Query user from database
    user = await get_user_by_username(db, username)
    if not user:
        return False
    if not verify_password(password, user.hashed_password):
        return False
    return user
```

FastAPI Security Dependencies

```python
# backend/security/dependencies.py
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from sqlalchemy.ext.asyncio import AsyncSession
from typing import Optional

from backend.database.session import get_db
from backend.security.jwt import decode_token
from backend.models.user import User
from backend.schemas.user import UserResponse

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
) -> User:
    payload = decode_token(token)
    username: str = payload.get("sub")
    if username is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid authentication credentials",
            headers={"WWW-Authenticate": "Bearer"},
        )

    user = await get_user_by_username(db, username)
    if user is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="User not found",
```

```
            headers={"WWW-Authenticate": "Bearer"},
        )
    return user

async def get_current_active_user(
    current_user: User = Depends(get_current_user)
) -> User:
    if not current_user.is_active:
        raise HTTPException(status_code=400, detail="Inactive user")
    return current_user

async def get_current_superuser(
    current_user: User = Depends(get_current_active_user)
) -> User:
    if not current_user.is_superuser:
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Not enough permissions"
        )
    return current_user
```

Authentication Router

```python
# backend/routers/auth.py
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.ext.asyncio import AsyncSession
from datetime import timedelta

from backend.database.session import get_db
from backend.security.auth import authenticate_user, get_password_hash
from backend.security.jwt import (
    create_access_token,
    create_refresh_token,
    ACCESS_TOKEN_EXPIRE_MINUTES,
    decode_token,
)
from backend.security.dependencies import get_current_user
from backend.models.user import User
from backend.schemas.user import UserCreate, UserResponse, Token, TokenRefresh

router = APIRouter(prefix="/api/auth", tags=["auth"])

@router.post("/register", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
async def register(user_data: UserCreate, db: AsyncSession = Depends(get_db)):
    # Check if user exists
    existing = await db.execute(
        select(User).where(
            (User.email == user_data.email) | (User.username == user_data.username)
        )
    )
    if existing.scalar_one_or_none():
        raise HTTPException(status_code=400, detail="User already exists")

    # Create user
    hashed_password = get_password_hash(user_data.password)
    db_user = User(
        username=user_data.username,
        email=user_data.email,
        hashed_password=hashed_password,
        full_name=user_data.full_name,
    )
    db.add(db_user)
    await db.commit()
    await db.refresh(db_user)
    return db_user

@router.post("/login", response_model=Token)
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: AsyncSession = Depends(get_db)
```

```
):
    user = await authenticate_user(db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token = create_access_token(data={"sub": user.username})
    refresh_token = create_refresh_token(data={"sub": user.username})

    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer"
    }

@router.post("/refresh", response_model=Token)
async def refresh_token(refresh_data: TokenRefresh):
    payload = decode_token(refresh_data.refresh_token)
    if payload.get("type") != "refresh":
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token type"
        )

    username = payload.get("sub")
    if not username:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid token"
        )

    access_token = create_access_token(data={"sub": username})
    return {
        "access_token": access_token,
        "refresh_token": refresh_data.refresh_token,
        "token_type": "bearer"
    }

@router.post("/logout")
async def logout(current_user: User = Depends(get_current_user)):
    # Invalidate token on client side
    return {"message": "Successfully logged out"}

@router.get("/me", response_model=UserResponse)
async def get_me(current_user: User = Depends(get_current_user)):
    return current_user
```

Flet Authentication Client

```python
# frontend/services/auth_service.py
import flet as ft
import httpx
import asyncio

class AuthService:
    def __init__(self, page: ft.Page):
        self.page = page
        self.base_url = "http://localhost:8000/api"
        self.token = None
        self.refresh_token = None
        self.user = None

    async def login(self, username: str, password: str) -> bool:
        try:
            async with httpx.AsyncClient() as client:
                response = await client.post(
                    f"{self.base_url}/auth/login",
                    data={"username": username, "password": password},
```

```
                headers={"Content-Type": "application/x-www-form-urlencoded"}
            )
            response.raise_for_status()
            data = response.json()
            self.token = data["access_token"]
            self.refresh_token = data["refresh_token"]

            # Save to client storage
            await self.page.client_storage.set("auth_token", self.token)
            await self.page.client_storage.set("refresh_token", self.refresh_token)

            await self.get_current_user()
            return True
        except Exception as e:
            print(f"Login error: {e}")
            return False

    async def get_current_user(self):
        if not self.token:
            return None

        try:
            async with httpx.AsyncClient() as client:
                response = await client.get(
                    f"{self.base_url}/auth/me",
                    headers={"Authorization": f"Bearer {self.token}"}
                )
                response.raise_for_status()
                self.user = response.json()
                return self.user
        except Exception as e:
            print(f"Error fetching user: {e}")
            return None

    async def logout(self):
        self.token = None
        self.refresh_token = None
        self.user = None
        await self.page.client_storage.remove("auth_token")
        await self.page.client_storage.remove("refresh_token")

    def is_authenticated(self) -> bool:
        return self.token is not None

    def get_auth_headers(self) -> dict:
        if self.token:
            return {"Authorization": f"Bearer {self.token}"}
        return {}

    async def authenticated_request(self, method: str, endpoint: str, **kwargs):
        if not self.token:
            raise Exception("Not authenticated")

        headers = kwargs.pop("headers", {})
        headers.update(self.get_auth_headers())

        try:
            async with httpx.AsyncClient() as client:
                response = await client.request(
                    method,
                    f"{self.base_url}{endpoint}",
                    headers=headers,
                    **kwargs
                )
                if response.status_code == 401:
                    # Try to refresh token
                    if await self.refresh_access_token():
                        headers.update(self.get_auth_headers())
                        response = await client.request(
                            method,
                            f"{self.base_url}{endpoint}",
                            headers=headers,
                            **kwargs
```

```
                )
                response.raise_for_status()
                return response.json()
        except Exception as e:
            print(f"Request error: {e}")
            raise

    async def refresh_access_token(self) -> bool:
        if not self.refresh_token:
            return False

        try:
            async with httpx.AsyncClient() as client:
                response = await client.post(
                    f"{self.base_url}/auth/refresh",
                    json={"refresh_token": self.refresh_token}
                )
                response.raise_for_status()
                data = response.json()
                self.token = data["access_token"]
                await self.page.client_storage.set("auth_token", self.token)
                return True
        except Exception:
            return False
```

Flet Login UI with Authentication

```python
# frontend/main.py
import flet as ft
import asyncio
from services.auth_service import AuthService

def main(page: ft.Page):
    page.title = "Secure App"
    page.theme_mode = ft.ThemeMode.LIGHT

    # Initialize auth service
    auth = AuthService(page)

    # Navigation function
    async def navigate_to(route: str):
        page.route = route
        await update_page()

    # Update page based on route
    async def update_page():
        page.views.clear()

        if not auth.is_authenticated():
            # Show login page
            page.views.append(create_login_view())
        else:
            # Show main app
            if page.route == "/profile":
                page.views.append(create_profile_view())
            else:
                page.views.append(create_main_view())

        page.update()

    def create_login_view():
        username_ref = ft.Ref[ft.TextField]()
        password_ref = ft.Ref[ft.TextField]()
        error_ref = ft.Ref[ft.Text]()

        async def handle_login(e):
            username = username_ref.current.value
            password = password_ref.current.value

            if not username or not password:
                error_ref.current.value = "Please fill in all fields"
```

```
            page.update()
            return

        success = await auth.login(username, password)
        if success:
            await navigate_to("/")
        else:
            error_ref.current.value = "Invalid credentials"
            page.update()

    return ft.View(
        route="/login",
        controls=[
            ft.SafeArea(
                content=ft.Column([
                    ft.Text("Welcome Back", size=32, weight=ft.FontWeight.BOLD),
                    ft.Text("Login to your account", size=16),
                    ft.TextField(
                        ref=username_ref,
                        label="Username",
                        prefix_icon=ft.Icons.PERSON,
                        width=300,
                    ),
                    ft.TextField(
                        ref=password_ref,
                        label="Password",
                        prefix_icon=ft.Icons.LOCK,
                        password=True,
                        can_reveal_password=True,
                        width=300,
                        on_submit=handle_login,
                    ),
                    ft.Text(ref=error_ref, color=ft.Colors.RED),
                    ft.Button(
                        "Login",
                        on_click=handle_login,
                        width=300,
                        height=50,
                    ),
                    ft.TextButton(
                        "Don't have an account? Register",
                        on_click=lambda e: page.launch_url("/register"),
                    ),
                ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=20),
            )
        ],
        vertical_alignment=ft.MainAxisAlignment.CENTER,
        horizontal_alignment=ft.CrossAxisAlignment.CENTER,
    )

def create_main_view():
    async def handle_logout(e):
        await auth.logout()
        await navigate_to("/login")

    return ft.View(
        route="/",
        controls=[
            ft.AppBar(
                title=ft.Text("Dashboard"),
                actions=[
                    ft.IconButton(
                        ft.Icons.PERSON,
                        on_click=lambda e: asyncio.create_task(navigate_to("/profile")),
                    ),
                    ft.IconButton(
                        ft.Icons.LOGOUT,
                        on_click=handle_logout,
                    ),
                ],
            ),
            ft.SafeArea(
                content=ft.Column([
```

```
                    ft.Text(f"Welcome, {auth.user['full_name'] or auth.user['username']}!", size=24),
                    ft.Card(
                        content=ft.Container(
                            content=ft.Column([
                                ft.Text("Your Data", size=20, weight=ft.FontWeight.BOLD),
                                ft.Text("This is protected content."),
                            ]),
                            padding=20,
                        ),
                    ),
                ], spacing=20),
            )
        ],
    )

    def create_profile_view():
        async def handle_back(e):
            await navigate_to("/")

        return ft.View(
            route="/profile",
            controls=[
                ft.AppBar(
                    title=ft.Text("Profile"),
                    leading=ft.IconButton(ft.Icons.ARROW_BACK, on_click=handle_back),
                ),
                ft.SafeArea(
                    content=ft.Column([
                        ft.CircleAvatar(
                            content=ft.Text(auth.user['full_name'][0] if auth.user['full_name'] else "U"),
                            radius=50,
                        ),
                        ft.Text(f"Username: {auth.user['username']}"),
                        ft.Text(f"Email: {auth.user['email']}"),
                        ft.Text(f"Full Name: {auth.user['full_name'] or 'Not set'}"),
                    ], horizontal_alignment=ft.CrossAxisAlignment.CENTER, spacing=20),
                )
            ],
        )

    # Handle route changes
    async def route_change(e):
        await update_page()

    page.on_route_change = route_change

    # Check for saved token
    async def check_auth():
        token = await page.client_storage.get("auth_token")
        if token:
            auth.token = token
            refresh = await page.client_storage.get("refresh_token")
            if refresh:
                auth.refresh_token = refresh
            await auth.get_current_user()

        await navigate_to(page.route or "/")

    asyncio.create_task(check_auth())

ft.run(main, view=ft.WEB_BROWSER, port=8550)
```

---

5. Background Tasks & Real-Time Events

FastAPI Background Tasks

```python
# backend/routers/tasks.py
from fastapi import APIRouter, BackgroundTasks, Depends
from fastapi_mail import FastMail, MessageSchema, ConnectionConfig
```

```
from pydantic import EmailStr
import time
import asyncio

router = APIRouter(prefix="/api/tasks", tags=["tasks"])

def send_email_task(email: str, subject: str, body: str):
    # Simulate email sending
    time.sleep(2)
    print(f"Email sent to {email}: {subject}")
    return True

def generate_report_task(user_id: int, report_type: str):
    # Simulate report generation
    time.sleep(5)
    print(f"Report generated for user {user_id}: {report_type}")
    return {"report_id": 123, "status": "completed"}

async def async_notification_task(user_id: int, message: str):
    # Async task example
    await asyncio.sleep(3)
    print(f"Notification sent to user {user_id}: {message}")
    return {"status": "notified"}

@router.post("/send-email")
async def send_email(
    email: str,
    subject: str,
    body: str,
    background_tasks: BackgroundTasks
):
    background_tasks.add_task(send_email_task, email, subject, body)
    return {"message": "Email scheduled for sending"}

@router.post("/generate-report/{user_id}")
async def generate_report(
    user_id: int,
    report_type: str,
    background_tasks: BackgroundTasks
):
    background_tasks.add_task(generate_report_task, user_id, report_type)
    return {"message": "Report generation started"}

@router.post("/notify/{user_id}")
async def send_notification(
    user_id: int,
    message: str,
    background_tasks: BackgroundTasks
):
    background_tasks.add_task(async_notification_task, user_id, message)
    return {"message": "Notification scheduled"}
```

Celery with Redis

```bash
# Install dependencies
uv add celery redis
```

```python
# backend/tasks/celery_app.py
from celery import Celery
import os

REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379/0")

celery_app = Celery(
    "tasks",
    broker=REDIS_URL,
    backend=REDIS_URL,
    include=["backend.tasks.worker_tasks"]
)
```

```
celery_app.conf.update(
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",
    timezone="UTC",
    enable_utc=True,
    task_track_started=True,
    task_time_limit=30 * 60,  # 30 minutes
    task_soft_time_limit=25 * 60,
    broker_connection_retry_on_startup=True,
)
```

```python
# backend/tasks/worker_tasks.py
from celery import Task
from backend.tasks.celery_app import celery_app
import time
import asyncio
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
import os

DATABASE_URL = os.getenv("DATABASE_URL")

@celery_app.task(bind=True, name="process_data")
def process_data(self, data_id: int, user_id: int):
    """Process data in background"""
    self.update_state(state="STARTED", meta={"progress": 0})

    # Simulate processing
    for i in range(10):
        time.sleep(1)
        self.update_state(state="PROGRESS", meta={"progress": (i + 1) * 10})

    result = {"data_id": data_id, "status": "completed", "processed_at": time.time()}
    self.update_state(state="SUCCESS", meta=result)
    return result

@celery_app.task(name="send_email_batch")
def send_email_batch(emails: list, subject: str, template: str):
    """Send emails in batch"""
    results = []
    for email in emails:
        # Simulate email sending
        time.sleep(0.5)
        results.append({"email": email, "status": "sent"})
    return {"total": len(results), "results": results}

@celery_app.task(name="cleanup_old_data")
def cleanup_old_data(days: int = 30):
    """Clean up old data"""
    time.sleep(10)
    return {"deleted": 100, "days": days}

@celery_app.task(name="generate_analytics_report")
def generate_analytics_report(start_date: str, end_date: str, report_type: str):
    """Generate analytics report"""
    time.sleep(15)
    return {
        "report_type": report_type,
        "start_date": start_date,
        "end_date": end_date,
        "total_records": 1000,
        "generated_at": time.time()
    }
```

```python
# backend/routers/celery_tasks.py
from fastapi import APIRouter, HTTPException
from backend.tasks.celery_app import celery_app
from backend.tasks.worker_tasks import process_data, send_email_batch, cleanup_old_data, generate_analytics_report
```

```
router = APIRouter(prefix="/api/celery", tags=["celery"])

@router.post("/process/{data_id}")
async def start_processing(data_id: int, user_id: int):
    task = process_data.delay(data_id, user_id)
    return {"task_id": task.id, "status": "queued"}

@router.get("/task/{task_id}")
async def get_task_status(task_id: str):
    task = celery_app.AsyncResult(task_id)

    if task.state == "PENDING":
        response = {"status": "pending", "progress": 0}
    elif task.state == "STARTED":
        meta = task.info
        response = {"status": "started", "progress": meta.get("progress", 0)}
    elif task.state == "SUCCESS":
        response = {"status": "completed", "result": task.result}
    elif task.state == "FAILURE":
        response = {"status": "failed", "error": str(task.info)}
    else:
        response = {"status": task.state}

    return response

@router.post("/email-batch")
async def send_batch_emails(emails: list, subject: str, template: str):
    task = send_email_batch.delay(emails, subject, template)
    return {"task_id": task.id, "status": "queued", "total": len(emails)}

@router.post("/cleanup")
async def cleanup_data(days: int = 30):
    task = cleanup_old_data.delay(days)
    return {"task_id": task.id, "status": "queued"}

@router.post("/analytics")
async def generate_report(start_date: str, end_date: str, report_type: str = "summary"):
    task = generate_analytics_report.delay(start_date, end_date, report_type)
    return {"task_id": task.id, "status": "queued"}
```

WebSocket for Real-Time Events

```python
# backend/routers/websocket.py
from fastapi import WebSocket, WebSocketDisconnect
from typing import Dict, Set
import json

class ConnectionManager:
    def __init__(self):
        self.active_connections: Dict[str, Set[WebSocket]] = {}
        self.general_connections: Set[WebSocket] = set()

    async def connect(self, websocket: WebSocket, room: str = "general"):
        await websocket.accept()
        if room == "general":
            self.general_connections.add(websocket)
        else:
            if room not in self.active_connections:
                self.active_connections[room] = set()
            self.active_connections[room].add(websocket)

    def disconnect(self, websocket: WebSocket, room: str = "general"):
        if room == "general":
            self.general_connections.discard(websocket)
        else:
            if room in self.active_connections:
                self.active_connections[room].discard(websocket)

    async def send_personal_message(self, message: str, websocket: WebSocket):
        await websocket.send_text(message)
```

```
    async def broadcast_to_room(self, message: dict, room: str = "general"):
        if room == "general":
            for connection in self.general_connections:
                await connection.send_text(json.dumps(message))
        else:
            if room in self.active_connections:
                for connection in self.active_connections[room]:
                    await connection.send_text(json.dumps(message))

    async def broadcast(self, message: dict):
        for connection in self.general_connections:
            await connection.send_text(json.dumps(message))
        for room_connections in self.active_connections.values():
            for connection in room_connections:
                await connection.send_text(json.dumps(message))

manager = ConnectionManager()

@router.websocket("/ws/{room}")
async def websocket_endpoint(websocket: WebSocket, room: str):
    await manager.connect(websocket, room)
    try:
        while True:
            data = await websocket.receive_text()
            message = json.loads(data)

            if message.get("type") == "message":
                await manager.broadcast_to_room(
                    {
                        "type": "message",
                        "user": message.get("user", "Anonymous"),
                        "content": message.get("content", ""),
                        "timestamp": datetime.now().isoformat(),
                    },
                    room
                )
            elif message.get("type") == "typing":
                await manager.broadcast_to_room(
                    {
                        "type": "typing",
                        "user": message.get("user", "Anonymous"),
                        "is_typing": message.get("is_typing", False),
                    },
                    room
                )
    except WebSocketDisconnect:
        manager.disconnect(websocket, room)
        await manager.broadcast_to_room(
            {
                "type": "user_left",
                "message": "A user has left the chat"
            },
            room
        )
```

Flet Real-Time WebSocket Client

```python
# frontend/services/websocket_service.py
import flet as ft
import websockets
import json
import asyncio
from typing import Callable, Optional

class WebSocketService:
    def __init__(self, page: ft.Page):
        self.page = page
        self.websocket: Optional[websockets.WebSocketClientProtocol] = None
        self.connected = False
        self.message_handlers: list[Callable] = []
```

```
    self.room = "general"
    self.username = "Anonymous"

async def connect(self, room: str = "general", username: str = "Anonymous"):
    self.room = room
    self.username = username

    try:
        self.websocket = await websockets.connect(f"ws://localhost:8000/ws/{room}")
        self.connected = True

        # Send join message
        await self.websocket.send(json.dumps({
            "type": "join",
            "user": username,
            "room": room
        }))

        # Start listening
        asyncio.create_task(self._listen())
        return True
    except Exception as e:
        print(f"WebSocket connection error: {e}")
        return False

async def _listen(self):
    try:
        while True:
            message = await self.websocket.recv()
            data = json.loads(message)

            # Notify handlers
            for handler in self.message_handlers:
                await handler(data)

    except websockets.exceptions.ConnectionClosed:
        self.connected = False
        print("WebSocket connection closed")
    except Exception as e:
        print(f"WebSocket error: {e}")
        self.connected = False

async def send_message(self, content: str):
    if not self.connected:
        return False

    try:
        await self.websocket.send(json.dumps({
            "type": "message",
            "user": self.username,
            "content": content
        }))
        return True
    except Exception as e:
        print(f"Error sending message: {e}")
        return False

async def send_typing(self, is_typing: bool = True):
    if not self.connected:
        return

    try:
        await self.websocket.send(json.dumps({
            "type": "typing",
            "user": self.username,
            "is_typing": is_typing
        }))
    except Exception:
        pass

def on_message(self, handler: Callable):
    self.message_handlers.append(handler)
```

```
    async def disconnect(self):
        self.connected = False
        if self.websocket:
            await self.websocket.close()

# frontend/views/chat_view.py
import flet as ft
from services.websocket_service import WebSocketService

def create_chat_view(page: ft.Page, ws: WebSocketService):
    messages_ref = ft.Ref[ft.Column]()
    input_ref = ft.Ref[ft.TextField]()
    typing_ref = ft.Ref[ft.Text]()
    status_ref = ft.Ref[ft.Text]()

    async def on_webSocket_message(data: dict):
        if data.get("type") == "message":
            messages_ref.current.controls.append(
                ft.Row([
                    ft.Text(f"{data.get('user', 'Unknown')}: ", weight=ft.FontWeight.BOLD),
                    ft.Text(data.get("content", "")),
                ])
            )
            messages_ref.current.update()
        elif data.get("type") == "typing":
            if data.get("is_typing", False):
                typing_ref.current.value = f"{data.get('user', 'Someone')} is typing..."
            else:
                typing_ref.current.value = ""
            typing_ref.current.update()
        elif data.get("type") == "user_left":
            messages_ref.current.controls.append(
                ft.Text(f"**{data.get('message', 'User left')}**", color=ft.Colors.GREY)
            )
            messages_ref.current.update()

    async def send_message(e):
        text = input_ref.current.value
        if text and ws.connected:
            await ws.send_message(text)
            input_ref.current.value = ""
            input_ref.current.update()

    async def connect_chat(e):
        status_ref.current.value = "Connecting..."
        status_ref.current.color = ft.Colors.ORANGE
        page.update()

        username = await page.client_storage.get("username") or "User"
        if await ws.connect("general", username):
            status_ref.current.value = "Connected"
            status_ref.current.color = ft.Colors.GREEN
            ws.on_message(on_webSocket_message)
        else:
            status_ref.current.value = "Connection failed"
            status_ref.current.color = ft.Colors.RED
        page.update()

    async def on_typing(e):
        if ws.connected:
            await ws.send_typing(True)

    async def on_stop_typing(e):
        if ws.connected:
            await ws.send_typing(False)

    return ft.Column([
        ft.Text("Real-Time Chat", size=32, weight=ft.FontWeight.BOLD),
        ft.Row([
            ft.Button("Connect", on_click=lambda e: asyncio.create_task(connect_chat(e))),
            ft.Text(ref=status_ref, value="Disconnected", color=ft.Colors.RED),
        ]),
        ft.Container(
```

```
            content=ft.Column(ref=messages_ref, scroll=ft.ScrollMode.AUTO),
            height=300,
            border=ft.Border.all(1, ft.Colors.GREY),
            border_radius=5,
            padding=10,
        ),
        ft.Text(ref=typing_ref, color=ft.Colors.GREY, italic=True),
        ft.Row([
            ft.TextField(
                ref=input_ref,
                hint_text="Type a message...",
                expand=True,
                on_focus=on_typing,
                on_blur=on_stop_typing,
                on_submit=lambda e: asyncio.create_task(send_message(e)),
            ),
            ft.IconButton(
                ft.Icons.SEND,
                on_click=lambda e: asyncio.create_task(send_message(e)),
            ),
        ]),
    ])
```

---

6. Packaging & Deployment

Docker Setup for Full-Stack App

Dockerfile for FastAPI Backend

```dockerfile
# Dockerfile.backend
FROM python:3.12-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    libpq-dev \
    && rm -rf /var/lib/apt/lists/*

# Copy dependencies
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev

# Copy application
COPY backend/ ./backend/
COPY shared/ ./shared/

ENV PYTHONPATH=/app
ENV DATABASE_URL=postgresql+asyncpg://user:password@db:5432/mydb

EXPOSE 8000

CMD ["uv", "run", "uvicorn", "backend.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile for Flet Frontend (Static Build)

```dockerfile
# Dockerfile.frontend
FROM python:3.12-slim

WORKDIR /app

# Install dependencies
COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen --no-dev

# Copy application
```

```
COPY frontend/ ./frontend/
COPY shared/ ./shared/

ENV PYTHONPATH=/app
ENV API_URL=http://backend:8000

EXPOSE 8550

CMD ["uv", "run", "python", "-m", "frontend.main"]
```

Docker Compose

```yaml
# docker-compose.yml
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  backend:
    build:
      context: .
      dockerfile: Dockerfile.backend
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    environment:
      DATABASE_URL: postgresql+asyncpg://user:password@db:5432/mydb
      REDIS_URL: redis://redis:6379/0
      SECRET_KEY: your-secret-key-here
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app/backend
      - ./shared:/app/shared

  celery_worker:
    build:
      context: .
      dockerfile: Dockerfile.backend
    command: uv run celery -A backend.tasks.celery_app worker --loglevel=info
    depends_on:
      - redis
      - db
    environment:
      DATABASE_URL: postgresql+asyncpg://user:password@db:5432/mydb
      REDIS_URL: redis://redis:6379/0
    volumes:
      - ./backend:/app/backend
```

```
      - ./shared:/app/shared

  frontend:
    build:
      context: .
      dockerfile: Dockerfile.frontend
    depends_on:
      - backend
    environment:
      API_URL: http://backend:8000
    ports:
      - "8550:8550"
    volumes:
      - ./frontend:/app/frontend
      - ./shared:/app/shared

volumes:
  postgres_data:
  redis_data:
```

Packaging Flet App for All Platforms

```bash
# Android
flet build apk --split-per-abi

# iOS (requires macOS)
flet build ipa

# Windows
flet build windows

# macOS
flet build macos

# Linux
flet build linux

# Web
flet build web

# All platforms with options
flet build apk --arch arm64-v8a x86_64 \
    --android-signing-key-store ~/upload-keystore.jks \
    --android-signing-key-alias upload
```

Deploy to Cloud Platforms

Render.com

```yaml
# render.yaml
services:
  - type: web
    name: fastapi-backend
    runtime: python
    buildCommand: pip install uv && uv sync
    startCommand: uv run uvicorn backend.main:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: DATABASE_URL
        fromDatabase:
          name: postgres-db
          property: connectionString
      - key: SECRET_KEY
        generateValue: true

  - type: web
    name: flet-frontend
    runtime: python
    buildCommand: pip install uv && uv sync
    startCommand: uv run python -m frontend.main
```

```
    envVars:
      - key: API_URL
        fromService:
          name: fastapi-backend
          type: web
          property: host

databases:
  - name: postgres-db
    databaseName: mydb
    user: user
```

Fly.io

```toml
# fly.toml (backend)
app = "myapp-backend"
primary_region = "sjc"

[build]
  dockerfile = "Dockerfile.backend"

[env]
  DATABASE_URL = "postgresql+asyncpg://..."
  SECRET_KEY = "..."

[[services]]
  http_checks = []
  internal_port = 8000
  processes = ["app"]
  protocol = "tcp"
  script_checks = []

  [services.concurrency]
    hard_limit = 25
    soft_limit = 20
    type = "connections"

  [[services.ports]]
    port = 80
    handlers = ["http"]
    force_https = true

  [[services.ports]]
    port = 443
    handlers = ["tls", "http"]

# fly.toml (frontend)
app = "myapp-frontend"
primary_region = "sjc"

[build]
  dockerfile = "Dockerfile.frontend"

[env]
  API_URL = "https://myapp-backend.fly.dev"

[[services]]
  internal_port = 8550
  protocol = "tcp"

  [[services.ports]]
    port = 80
    handlers = ["http"]
    force_https = true

  [[services.ports]]
    port = 443
    handlers = ["tls", "http"]
```

---

7. Complete Full-Stack Application Example

Project Structure

```
fullstack_app/
├── frontend/
│   ├── main.py
│   ├── views/
│   │   ├── login.py
│   │   ├── dashboard.py
│   │   └── profile.py
│   └── services/
│       ├── api.py
│       └── auth.py
├── backend/
│   ├── main.py
│   ├── models/
│   │   └── __init__.py
│   ├── schemas/
│   │   └── __init__.py
│   ├── routers/
│   │   ├── auth.py
│   │   ├── users.py
│   │   └── items.py
│   ├── database/
│   │   └── session.py
│   └── security/
│       └── __init__.py
├── shared/
│   └── models.py
├── pyproject.toml
├── docker-compose.yml
└── .env
```

Shared Models

```python
# shared/models.py
from pydantic import BaseModel, EmailStr, Field
from datetime import datetime
from typing import Optional, List, Any

class UserBase(BaseModel):
    username: str = Field(..., min_length=3, max_length=50)
    email: EmailStr
    full_name: Optional[str] = None

class UserCreate(UserBase):
    password: str = Field(..., min_length=8)

class UserResponse(UserBase):
    id: int
    is_active: bool
    created_at: datetime

class ItemBase(BaseModel):
    name: str = Field(..., min_length=1, max_length=100)
    description: Optional[str] = None
    price: float = Field(..., gt=0)

class ItemCreate(ItemBase):
    pass

class ItemResponse(ItemBase):
    id: int
    owner_id: int
    created_at: datetime

class Token(BaseModel):
    access_token: str
```

```
    refresh_token: str
    token_type: str = "bearer"

class TokenRefresh(BaseModel):
    refresh_token: str

class ApiResponse(BaseModel):
    status: str
    message: str
    data: Optional[Any] = None
```

Complete Backend

```python
# backend/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager
from backend.database.session import engine, Base
from backend.routers import auth, users, items

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Startup
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield
    # Shutdown
    await engine.dispose()

app = FastAPI(
    title="Full-Stack API",
    version="1.0.0",
    lifespan=lifespan,
)

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:8550", "http://localhost:8000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Include routers
app.include_router(auth.router)
app.include_router(users.router)
app.include_router(items.router)

@app.get("/api/health")
async def health_check():
    return {"status": "healthy", "timestamp": datetime.now().isoformat()}

# Run: uvicorn backend.main:app --reload --port 8000
```

Complete Flet Frontend

```python
# frontend/main.py
import flet as ft
import asyncio
from services.auth import AuthService
from views.login import LoginView
from views.dashboard import DashboardView
from views.profile import ProfileView

async def main(page: ft.Page):
    page.title = "Full-Stack App"
    page.theme_mode = ft.ThemeMode.LIGHT
    page.adaptive = True
```

```
    auth = AuthService(page)

    async def navigate_to(route: str):
        page.route = route
        await update_page()

    async def update_page():
        page.views.clear()

        if not auth.is_authenticated():
            page.views.append(LoginView(page, auth, navigate_to).build())
        else:
            if page.route == "/profile":
                page.views.append(ProfileView(page, auth, navigate_to).build())
            elif page.route == "/items":
                page.views.append(DashboardView(page, auth, navigate_to).build())
            else:
                page.views.append(DashboardView(page, auth, navigate_to).build())

        page.update()

    async def route_change(e):
        await update_page()

    page.on_route_change = route_change

    # Check saved auth
    token = await page.client_storage.get("auth_token")
    if token:
        auth.token = token
        refresh = await page.client_storage.get("refresh_token")
        if refresh:
            auth.refresh_token = refresh
        await auth.get_current_user()

    await update_page()

if __name__ == "__main__":
    ft.run(main, view=ft.WEB_BROWSER, port=8550)
```

Conclusion

You now have a comprehensive understanding of building full-stack applications with Flet and FastAPI:

Key Takeaways

1. **Integration Options**:
   - Mount Flet directly in FastAPI (ASGI)
   - Run separate services with HTTP communication
   - Use WebSockets for real-time updates

2. **Database**:
   - SQLAlchemy for async ORM
   - SQLModel for simplified models
   - Alembic for migrations

3. **Authentication**:
   - JWT tokens with refresh
   - Password hashing with bcrypt
   - Secure session management

4. **Real-Time Features**:
   - WebSocket communication
   - Background tasks (FastAPI & Celery)
   - Pub/Sub patterns

5. **Deployment**:
   - Docker containerization
   - Cloud platforms (Render, Fly.io)

- Multi-platform packaging with Flet

Next Steps

1. Build a small project to practice
2. Add more features (notifications, file uploads, etc.)
3. Implement proper logging and monitoring
4. Add CI/CD pipeline
5. Scale horizontally with load balancers

End of Complete Full-Stack Guide